

使用 C# 建構 gRPC 服務

來源：<https://codelabs.developers.google.com/codelabs/cloud-grpc-csharp?hl=zh-tw>

步驟數：7

在本程式碼研究室中，您將學習如何建構 C# 服務，以便透過 gRPC 公開 API，並建立 C# 用戶端來呼叫 gRPC 服務。

目錄

1. [1. 總覽](#)
2. [2. 下載並建構 gRPC C# 範例](#)
3. [3. 探索 Greeter 範例](#)
4. [4. 執行 Greeter 範例](#)
5. [5. 更新 Greeter 範例](#)
6. [6. 探索 Chat 範例](#)
7. [7. 恭喜 !](#)

1. 總覽

gRPC 是 Google 開發的語言和平台中立遠端程序呼叫 (RPC) 架構和工具組。您可以使用通訊協定緩衝區定義服務，這是一套功能強大的二進位序列化工具集和語言。然後，您就可以從各種語言的服務定義中，產生慣用的用戶端和伺服器存根。

在本程式碼研究室中，您將瞭解如何使用 gRPC 架構建構公開 API 的 C# 服務。您可以使用以 C# 編寫的控制台用戶端與這項服務互動，該用戶端與服務使用相同的服務說明。

課程內容

- 通訊協定緩衝區語言。
- 如何使用 C# 實作 gRPC 服務。
- 如何使用 C# 實作 gRPC 用戶端。
- 如何更新 gRPC 服務。

軟硬體需求

- [Chrome](#) 或 [Firefox](#) 瀏覽器。
 - 已安裝 [Visual Studio 2013](#) 或以上版本。
 - 熟悉 .NET Framework 和 [C# 語言](#)。
-

2. 下載並建構 gRPC C# 範例

下載範例

以 ZIP 檔案格式[下載 gRPC C# 範例](#)存放區，然後解壓縮。

或者，您也可以複製其 `git` 存放區。

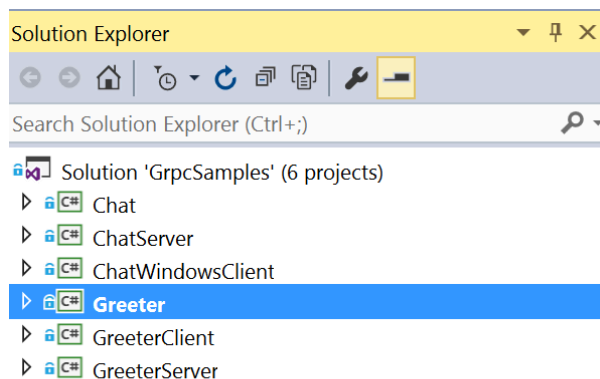
```
git clone https://github.com/meteatamel/grpc-samples-dotnet.git
```

無論採用哪種方式，您都應該會有一個 `grpc-samples-dotnet` 資料夾，內含下列內容：

☐ Name	Date modified	Type	Size
Chat	28-Apr-17 07:33	File folder	
ChatServer	28-Apr-17 07:33	File folder	
ChatWindowsClient	28-Apr-17 07:33	File folder	
Greeter	28-Apr-17 07:33	File folder	
GreeterClient	28-Apr-17 07:33	File folder	
GreeterServer	28-Apr-17 07:33	File folder	
CONTRIBUTING.md	28-Apr-17 07:33	Markdown Source...	1 KB
generate_protos.bat	28-Apr-17 07:33	Windows Batch File	3 KB
GrpcSamples.sln	28-Apr-17 07:33	Microsoft Visual S...	4 KB
LICENSE	28-Apr-17 07:33	File	12 KB
README.md	28-Apr-17 07:33	Markdown Source...	1 KB

建構解決方案

範例應用程式資料夾包含 Visual Studio 解決方案 `GrpcSamples.sln`。按兩下即可在 Visual Studio 中開啟解決方案。您應該會看到包含多個專案的解決方案。



我們會在下一節中詳細說明專案，但首先，請先建構專案。前往 `Build > Build Solution` 或 `Ctrl + Shift + B` 建構解決方案。這會從 NuGet 下載所有依附元件，然後編譯應用程式。

最後，您應該會在 Visual Studio 控制台輸出中看到 `Build succeeded` 訊息。

3. 探索 Greeter 範例

Greeter 是簡單的 gRPC 範例，用戶端會傳送含有名稱的要求，伺服器則會傳送訊息和名稱做為回應。Greeter 專案包含用戶端和伺服器所依據的通用服務定義 (proto 檔案)。

Greeter 專案

這是用戶端和伺服器共用的專案。其中包含 `greeter.proto`，這是用戶端和伺服器使用的 gRPC 服務定義。服務定義會定義名為 `GreetingService` 的 gRPC 服務，並具有 `greeting` 方法，該方法會將 `HelloRequest` 做為輸入內容，並將 `HelloResponse` 做為輸出內容。

```
service GreetingService {  
    rpc greeting>HelloRequest) returns (HelloResponse);  
}
```

這是 unary (即無串流) 方法，用戶端會傳送單一要求，並從伺服器取得單一回應。您可以瀏覽 `greeter.proto` 的其餘內容。這個專案也有名為 `generate_protos.bat` 的指令碼，可用於從 proto 檔案產生用戶端和伺服器虛設常式。專案已包含產生的用戶端和伺服器存根，因此您不必自行產生。不過，如果變更服務定義檔的內容，就必須執行這個指令碼，重新產生存根。

Greeter Server

這是 gRPC 伺服器的專案。`Program.cs` 是主要進入點，用於設定通訊埠和伺服器實作。重要類別為 `GreeterServiceImpl.cs`。其中包含 `greeting` 方法，可實作實際功能。

```
public override Task<HelloResponse> greeting>HelloRequest request,  
    ServerCallContext context)  
{  
    return Task.FromResult(new HelloResponse {  
        Greeting = "Hello " + request.Name });  
}
```

迎賓人員用戶端

這是 gRPC 服務的用戶端。此外，它也將 `Program.cs` 設為進入點。這會建立與伺服器通訊的管道，然後使用產生的存根，透過該管道建立用戶端。接著，它會建立要求，並使用用戶端存根傳送至伺服器。

步驟 4 · 約 5 分鐘

4. 執行 Greeter 範例

首先，啟動 Greeter 伺服器。開啟命令提示字元，然後前往 Greeter Server 的 `bin > Debug` 資料夾並執行可執行檔。您應該會看到伺服器正在接聽。

```
> C:\grpc-samples-dotnet\GreeterServer\bin\Debug>GreeterServer.exe
GreeterServer listening on port 50051
Press any key to stop the server...
```

接著，執行 Greeter Client。在另一個命令提示字元中，前往 Greeter Server 的 `bin > Debug` 資料夾，然後執行可執行檔。您應該會看到用戶端傳送要求，並收到伺服器的回應。

```
> C:\grpc-samples-dotnet\GreeterClient\bin\Debug>GreeterClient.exe
GreeterClient sending request
GreeterClient received response: Hello Mete - on C#
Press any key to exit...
```

步驟 5 · 約 5 分鐘

5. 更新 Greeter 範例

我們來看看更新服務的樣子。在 gRPC 服務中新增名為 goodbye 的方法，向用戶端傳回 goodbye 而非 hello。

第一步是更新服務定義檔 `greeter.proto`。

```
service GreetingService {  
    rpc greeting>HelloRequest) returns (HelloResponse);  
  
    rpc goodbye>HelloRequest) returns (HelloResponse);  
}
```

接著，您需要重新產生用戶端和伺服器虛設常式。在命令提示字元中執行 `generate_protos.bat`。產生存根後，您可能需要重新整理 Visual Studio 專案，才能取得更新的程式碼。

最後，更新用戶端和伺服器程式碼，充分運用新方法。在服務中更新 `GreeterServiceImpl.cs`，並新增 `goodbye` 方法。

```
public override Task<HelloResponse> goodbye>HelloRequest request,  
    ServerCallContext context)  
{  
    return Task.FromResult(new HelloResponse {  
        Greeting = "Goodbye " + request.Name });  
}
```

在用戶端中，呼叫 `Program.cs` 中的 `goodbye` 方法

```
response = client.goodbye(request);  
Console.WriteLine("GreeterClient received response: "  
    + response.Greeting);
```

重新建構專案，然後再次執行伺服器和用戶端。這時用戶端應該會收到再見訊息。

```
> C:\grpc-samples-dotnet\GreeterClient\bin\Debug>GreeterClient.exe  
GreeterClient sending request  
GreeterClient received response: Hello Mete - on C#  
GreeterClient received response: Goodbye Mete - on C#  
Press any key to exit...
```


6. 探索 Chat 範例

解決方案中也有 `ChatServer` 和 `ChatWindowsClient` 專案。顧名思義，這是簡易聊天應用程式的用戶端和伺服器配對。`Chat` 專案有稱為 `chat.proto` 的服務定義檔。其中定義了 `ChatService` 和 `chat` 方法。

```
service ChatService {  
  rpc chat(stream ChatMessage) returns (stream ChatMessageFromServer);  
}
```

重點是傳入和傳出的即時通訊訊息都會標上「`stream`」關鍵字。這基本上會將連線變成雙向串流，用戶端和伺服器可以隨時互傳訊息，是聊天應用程式的完美解決方案。

您可以進一步探索範例、建構及執行範例，藉此練習瞭解運作方式。

步驟 7 · 約 1 分鐘

7. 恭喜！

涵蓋內容

- 通訊協定緩衝區語言。
- 如何使用 C# 實作 gRPC 服務。
- 如何使用 C# 實作 gRPC 用戶端。
- 如何更新 gRPC 服務。

後續步驟

- 進一步瞭解 [Google Cloud Platform 上的 Windows](#)。
 - 進一步瞭解 [Google Cloud Platform 上的 .NET](#)。
 - 進一步瞭解 [Cloud Tools for Visual Studio](#)。
-